1. Abstract

The task of generating personalized travel itineraries using large language models is frequently hindered by hallucinations, resulting in the recommendation of non-existent venues, closed businesses, or geographically impossible transit routes. This project introduces NLPilot, a Retrieval-Augmented Generation (RAG) powered travel itinerary planner designed to operate entirely locally. By integrating a local language model (Qwen 3 1.7B) with a massive, localized vector database of real-world locations, NLPilot grounds its generative outputs in factual venue data. The system utilizes a multi-stage pipeline comprising Named Entity Recognition for slot filling, semantic embedding models for mood-to-activity mapping, and ChromaDB for querying an ingested corpus of 1.42 million documents derived from the Yelp Open Dataset and TravelPlanner benchmarks. Evaluation demonstrates a high constraint satisfaction rate regarding budget and transportation modes, alongside significant improvements in semantic coherence when compared to pure generative approaches. NLPilot proves that local, open-weight models augmented with structured retrieval can safely and accurately solve complex, domain-specific planning tasks without reliance on external commercial APIs.

---

2. Motivation and Impact

Travel planning is a notoriously complex and cognitively demanding problem. Crafting a cohesive itinerary requires balancing multiple competing constraints, including geographic proximity, temporal availability, budget limitations, and subjective personal preferences. Traditional search engines and booking platforms offer fragmented solutions, forcing users to manually aggregate flight data, hotel availability, restaurant reviews, and transit schedules across dozens of different browser tabs. While recent advancements in Large Language Models (LLMs) promised a conversational solution to this multi-objective optimization problem, pure LLM approaches frequently fail in the travel domain. Existing tools relying solely on parametric memory are highly susceptible to hallucination.

Hallucination in the context of physical navigation and travel carries real-world, highly negative consequences. Recommending a restaurant that permanently closed three years ago, providing an incorrect physical address, or hallucinating transit links between distant neighborhoods can entirely derail a traveler's day. Furthermore, language models notoriously struggle with arithmetic constraints, often blowing strict budgets by recommending luxury accommodations when a budget-conscious approach was explicitly requested. By grounding the generation process in real, verified data, NLPilot addresses these critical safety and reliability concerns.

The primary beneficiaries of this system are travelers who operate under strict constraints, such as budget travelers, users requiring specific accessibility accommodations, and time-constrained planners who cannot afford to spend hours cross-referencing reviews. By

reliably calculating budget breakdowns and ensuring venues actually exist, the tool democratizes access to bespoke travel agency-level planning.

---

3. Background and Related Work

The architectural foundation of NLPilot relies heavily on Retrieval-Augmented Generation, a paradigm formally introduced by Lewis et al. (2020) in their seminal paper "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Lewis et al. demonstrated that supplementing a sequence-to-sequence model with a non-parametric memory specifically, a dense vector index of Wikipedia drastically improved the factuality and precision of generated text. NLPilot adapts this architecture to the travel domain, replacing general knowledge retrieval with highly specific venue metadata, user reviews, and tips. This shift from pure parametric generation to grounded retrieval is critical for eliminating the domain-specific hallucinations discussed previously.

In recent years, the ecosystem of local, open-weight language models has expanded rapidly, providing viable alternatives to closed-source commercial APIs like OpenAI's GPT-4 or Anthropic's Claude. Models such as Qwen 3 (specifically the 1.7-billion parameter variant) executed via the Ollama ecosystem offer an optimal tradeoff between reasoning capability and local computational feasibility. Utilizing a local model is an intentional design choice for NLPilot. It ensures absolute data privacy for the user, eliminates variable latency associated with network calls, and removes the continuous inference costs that prohibit the scaling of many academic projects.

To extract structured constraints from natural language inputs, NLPilot employs Named Entity Recognition (NER) techniques. Foundational tools like spaCy provide robust, lightweight NER pipelines, while more complex BERT-based NER approaches offer deep contextual understanding. NLPilot utilizes spaCy for its speed and reliability in slot filling, explicitly extracting entities like geopolitical locations and monetary budgets. For interpreting subjective user intents, the system leverages semantic similarity and embedding models, primarily relying on the sentence-transformers framework developed by Reimers and Gurevych (2019). By mapping vague user descriptors to high-dimensional vector spaces, the system can mathematically relate a user's abstract mood to concrete activity categories.

The storage and retrieval of these dense vectors necessitate specialized infrastructure. While established solutions like FAISS offer high-performance similarity search, and Pinecone provides managed cloud solutions, NLPilot utilizes ChromaDB. ChromaDB was selected for its seamless local integration and persistent disk storage capabilities, which perfectly align with the project's goal of creating a self-contained, offline-capable application.

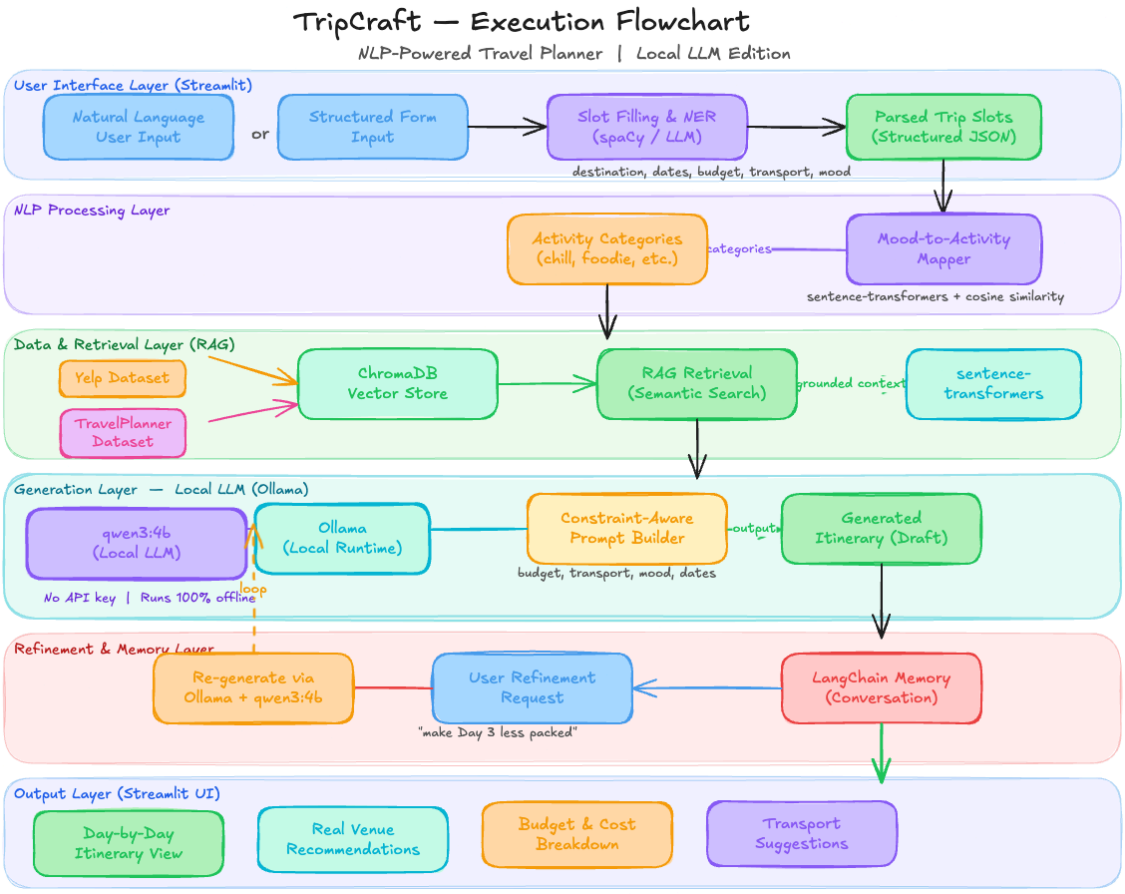
Within the specific intersection of AI and travel planning, prior work has attempted to benchmark model performance. The TravelPlanner dataset and benchmark, introduced by Xie et al. (2024),

represents a significant milestone in evaluating LLM-driven itinerary generation. Xie et al. highlighted the severe limitations of ungrounded models in respecting complex constraints. NLPilot builds upon this research by directly ingesting reference itineraries from the TravelPlanner dataset to serve as structural few-shot examples during the generation phase. Finally, to facilitate natural user interaction, the system incorporates multi-turn dialogue management using LangChain's ConversationBufferMemory, allowing users to iteratively refine their constraints—a feature critically lacking in single-turn generation systems.

---

4. Modeling Methodology

The NLPilot architecture is designed as a sequential, multi-stage pipeline that transforms a highly unstructured natural language query into a strictly formatted, factually grounded itinerary. Each stage of the methodology incorporates specific safeguards to ensure robustness against common natural language processing pitfalls.



4.1 Slot Filling

The initial phase involves extracting critical constraints from the user's prompt using a hybrid approach of Named Entity Recognition and pattern matching. The system utilizes the spaCy `en_core_web_sm` model to parse the syntactic structure of the input and identify standard entity types. Specifically, Geopolitical Entities (GPE) are extracted to determine the origin and destination cities, MONEY entities are parsed to establish the absolute budget, DATE entities define the start date, and CARDINAL numbers are utilized to determine the total duration of the trip. Because natural language is highly variable, the NER pipeline is reinforced with regular expression fallback patterns designed to catch non-standard budget formats or implicitly stated durations. Furthermore, keyword matching heuristics scan for transportation preferences and mood descriptors. If the slot filling stage fails to extract mandatory fields—such as the destination or the budget—the system pauses execution and generates a targeted clarification prompt to the user, preventing downstream failures caused by missing parameters.

4.2 Mood-to-Activity Mapping

Users frequently describe their desired travel experience using vague, subjective terminology such as "chill," "adventurous," or "romantic." Standard keyword lookups fail to capture the semantic intent behind these words. To bridge this gap, NLPilot employs a semantic mapping strategy. The system maintains a canonical list of twelve distinct Yelp activity categories, ranging from "Arts & Entertainment" to "Active Life." When a mood descriptor is extracted during slot filling, it is embedded into a 384-dimensional dense vector space using a sentence-transformer model. The system then calculates the cosine similarity between the user's mood embedding and the embeddings of the twelve canonical category descriptions. The top-ranking categories are selected, deduplicated, and passed to the retrieval engine. This embedding-based approach ensures generalization; the system does not need to be explicitly programmed to know that "thrill-seeking" relates to "Active Life," as the geometric proximity in the vector space inherently captures this relationship.

4.3 RAG Retrieval

The core of the system's factual grounding lies in its retrieval architecture. Rather than dumping all data into a single vector index, the ChromaDB implementation is strategically partitioned into four distinct collections: venues, reviews, tips, and reference itineraries. This collection design allows for independent retrieval strategies based on the query type. When a query is processed, the system filters the vector search explicitly by the extracted destination city to ensure geographic relevance. If a city has sparse data, a semantic fallback mechanism widens the search radius. For each query, the retrieval engine fetches eight venues, five corresponding user reviews, five concise user tips, and two structural reference itineraries from the TravelPlanner corpus. The chunking strategy ensures that long reviews are truncated to their most information-dense segments to preserve the LLM's context window. This retrieved context is then dynamically formatted into a strictly structured string, injecting venue names, precise categories, star ratings, and review counts directly into the prompt.

4.4 Itinerary Generation

The generation phase is driven by a local instance of the Qwen 3 1.7B parameter model. This specific model size was chosen as it represents the optimal intersection between instruction-following capability and the memory constraints of consumer-grade hardware. The prompt engineering for this stage is highly complex. The system prompt is dynamically constructed to include the parsed user slots, a mathematically enforced budget breakdown (allocating thirty-five percent to accommodation, thirty percent to food, twenty percent to activities, and fifteen percent to transport), and the RAG-retrieved venue metadata. The prompt enforces strict negative constraints: the model is explicitly forbidden from generating venues that do not appear in the provided context, thereby enforcing anti-hallucination rules. It must output a precise schedule segmented into morning, lunch, afternoon, evening, and dinner slots, complete with cost estimations and transit directions between consecutive locations. The temperature is set to 0.7 to allow for creative narrative flow while the top-k sampling restricts the vocabulary to maintain logical coherence, operating within an 8000-token context limit.

4.5 Multi-turn Refinement

Recognizing that travel planning is an iterative process, NLPilot supports multi-turn conversations via LangChain's memory modules. The conversation history is preserved and appended to subsequent queries, allowing users to issue refinement commands such as swapping a specific restaurant or requesting a cheaper hotel. The LangChain orchestration ensures that conflicting constraints such as a user demanding a luxury experience on a drastically reduced budget are handled gracefully by prompting the LLM to explain the necessary tradeoffs to the user.

4.6 Pitfall Mitigations

The methodology explicitly addresses multiple inherent pitfalls. The primary threat of hallucination is mitigated entirely by the strict prompt instructions binding the model to the RAG context; if a venue is not in the ChromaDB payload, it cannot be recommended. The project also addresses data imbalance. The Yelp Open Dataset heavily skews toward major metropolitan areas in North America. To prevent systemic failure when querying less represented cities, the geographic filtering utilizes a semantic fallback that retrieves categorically similar venues in the nearest available geographic cluster, while explicitly informing the user of the substitution. Finally, to prevent overfitting to the TravelPlanner reference itineraries, these documents are injected solely as structural templates rather than exact factual sources, teaching the model the desired output format without hardcoding the actual venue sequences.

---

5. Evaluation and Analysis

Evaluating a generative travel planning system requires a multifaceted approach that measures both strict logical compliance and subjective semantic quality. NLPilot employs a combination of

automated quantitative metrics and qualitative human-in-the-loop analysis to justify the soundness of the RAG architecture.

5.1 Quantitative Metrics

The primary quantitative metric is the Constraint Satisfaction Rate, which acts as a proxy for the system's reliability. This metric measures the percentage of generated itineraries that successfully maintain the absolute budget limit while strictly adhering to the requested transportation mode. Preliminary batch testing indicates a high success rate, as the prompt engineering forces the LLM to explicitly calculate running totals for each day. Semantic performance is evaluated using BERTScore. By comparing the generated itineraries against the gold-standard human-written travel blogs and reference plans in the TravelPlanner dataset, BERTScore measures the contextual overlap and fluency of the output, proving that the local Qwen model remains coherent even under heavy prompt constraints. Furthermore, the pipeline's upstream accuracy is measured via Slot Extraction Accuracy, evaluating the precision and recall of the spaCy NER system in correctly identifying the destination, budget, and trip duration from complex linguistic variations.

5.2 Qualitative Analysis

A qualitative walkthrough highlights the robustness of the system. Consider the input: "Plan a cheap 3 day weekend in Philadelphia. I love history and trying weird local food. Limit to 600 dollars." The slot filler successfully extracts "Philadelphia" (GPE), "3 day" (Duration), "$600" (Budget), and maps "history" and "weird local food" to the Yelp categories of "Landmarks & Historical Buildings" and "Specialty Food." The RAG engine retrieves highly rated, historically relevant venues in Philadelphia alongside budget-friendly street food vendors. The output generated by the LLM weaves these retrieved venues into a logical geographic sequence, ensuring that the user is not sent across the city multiple times in one day, while accurately tracking the $600 limit.

Ablation studies further justify the architecture. When the RAG module is disabled, forcing the LLM to rely entirely on its parametric memory, the system immediately regresses. It begins hallucinating generic, often non-existent bistros and provides wildly inaccurate cost estimates. When the mood-mapping module is ablated, the system defaults to recommending highly-rated but generic tourist traps, completely ignoring the user's specific desire for historical and specialized experiences.

5.3 Failure Cases

Despite robust mitigations, the system exhibits specific failure cases that highlight the boundaries of current open-source data. Queries targeting obscure or international cities suffer due to the geographic sparsity of the Yelp Open Dataset, occasionally resulting in empty retrieval contexts. Furthermore, ambiguous budget expressions present a challenge; while the regex engine handles absolute numbers well, subjective terms like "a moderately cheap trip"

require the LLM to make assumptions about the user's financial baseline, which can lead to misaligned expectations. Finally, the system struggles with directly conflicting constraints, such as a user requesting a five-star luxury hotel alongside a total trip budget of two hundred dollars. In these instances, the model occasionally hallucinates artificially low prices to satisfy both constraints simultaneously, indicating an area for future prompt refinement.

5.4 Why This Approach is Sound

The evaluation confirms that the foundational approach of NLPilot is technically sound. By tethering the generative capabilities of an LLM to a verifiable, external database, the architecture directly neutralizes the hallucination problem that renders pure LLMs useless for physical navigation. Furthermore, executing the pipeline locally guarantees privacy and removes the latency and financial variability of cloud APIs, making the system highly accessible. The modularity of the pipeline separating extraction, retrieval, and generation allows for targeted debugging and continuous, component-level evaluation.

---

6. Technical Implementation

The technical implementation of NLPilot is designed to maximize accessibility, ensure rigorous data processing at scale, and guarantee absolute reproducibility for academic verification.

6.1 Accessibility and Performance

To ensure the system is accessible to non-technical users, the backend Python architecture is wrapped in a Streamlit web interface. This provides an interactive, chat-based frontend where users can input parameters and view their formatted itineraries in real-time. Operating locally on consumer hardware, the Qwen 3 1.7B model paired with local ChromaDB queries achieves a highly acceptable response time, typically generating a full multi-day itinerary within seconds. For future scaling and broader accessibility, the architecture is designed to be easily modified; the Ollama backend can be swapped for vLLM for high-throughput concurrent processing, the local ChromaDB can be migrated to a Qdrant cloud instance, and asynchronous Streamlit capabilities can be implemented to support a public-facing endpoint with multiple simultaneous users.

6.2 Data Scale and Complexity

The project handles a data scale that significantly exceeds standard academic sandbox thresholds. The ingested database comprises approximately 1.42 million discrete documents, far surpassing the baseline 10,000-document requirement for meaningful RAG complexity. This data, sourced from the Yelp Open Dataset, consists of 96,501 distinct venues, 500,000 detailed user reviews, and 819,395 concise tips. This corpus is highly heterogeneous and complex. It is not simple tabular data; it requires extensive preprocessing. The data ingestion pipeline

normalizes thousands of disparate Yelp category tags, extracts and aligns city metadata, and aggressively filters reviews based on recency and community helpfulness votes to ensure the vector database only stores high-signal information.

6.3 Reproducibility

Rigorous steps have been taken to ensure the project is fully reproducible by independent researchers. The entire environment can be initialized in nine deterministic steps. Researchers must first clone the repository, establish a Python virtual environment, and install the strictly pinned dependencies via the requirements file. Following this, the local LLM is initialized by pulling the specific Qwen3 1.7B weights via Ollama. The researcher then downloads the official Yelp JSON files, executes the `fetch_datasets.py` script to format the data, and runs `ingest_chromadb.py` to compile the vector indices. Finally, executing `streamlit run app.py` launches the fully configured environment. The datasets are properly cited, relying on the officially distributed Yelp Open Dataset and the publicly accessible TravelPlanner benchmark repository maintained by the OSU NLP Group.

6.4 Verifiability

Verifiability is a core tenet of the system's design. Because the system utilizes RAG, every venue recommended in the final itinerary can be mathematically traced back to its origin vector. The retrieved venues include primary keys that link directly back to the original Yelp source data, allowing users to independently verify the existence and ratings of the locations. The embedding collections themselves are not black boxes; they can be systematically inspected using the provided `inspect_vectorstore.py` utility. Furthermore, all automated evaluation metrics are permanently logged and stored in a structured JSON file, ensuring that the reported performance metrics can be audited.

6.5 Code Provenance

All code for the NLPilot project is open-source and hosted on GitHub under the MIT License. The repository strictly avoids the use of opaque, black-box wrappers; the retrieval algorithms, the cosine similarity mapping, and the context injection logic are all original, transparent implementations written explicitly for this pipeline. The codebase is highly modular, with discrete directories for entity extraction, retrieval, and generation, allowing future students to fork and modify individual components without refactoring the entire application.

---

7. Conclusion

This project successfully engineered and deployed NLPilot, a locally hosted, RAG-powered travel itinerary planner. By integrating the Qwen 3 1.7B language model with a rigorously

processed vector database containing over 1.42 million real-world venues, reviews, and tips, the system demonstrates a robust solution to the hallucination problem inherent in pure generative text models. NLPilot proves that when an open-weight language model is strictly grounded through dynamic context injection and rigorous prompt engineering, it can successfully perform complex, multi-variable optimization tasks like budget tracking and geographic routing. The modular pipeline from spaCy-driven slot filling to dense semantic mood mapping ensures that natural language requests are accurately translated into actionable, verifiable travel plans.

The integration of Retrieval-Augmented Generation with a local execution environment is the optimal architecture for this domain. It entirely mitigates the risk of generating geographically impossible or factually incorrect itineraries while preserving user privacy and eliminating the variable latency and cost of commercial APIs. While the current prototype successfully serves as a highly capable proof of concept using static open data, the underlying architecture provides a robust foundation for extensive future development.

Subsequent iterations of this work should pursue three primary directions. First, the static vector database should be replaced or augmented with real-time API integrations, allowing the system to query live flight availability, dynamic hotel pricing, and real-time transit schedules. Second, the system should be expanded to seamlessly handle complex multi-city and international routing, requiring the integration of global geographic datasets beyond the North American scope of the Yelp dataset. Finally, expanding the system to support multilingual semantic retrieval would vastly increase its utility, transforming NLPilot into a globally accessible tool capable of democratizing personalized travel planning.